

Four on the Floor: Controlling Robotic Percussion with a Neural Network

ECE 490/590

Final Report

Elijah Verhoff

1. Overview

This project implements a neural network that performs real-time beat tracking on audio, with the downstream goal of controlling a mechanical actuator to “keep time” with music. The core technical contribution is a causal temporal convolutional network (TCN) that processes log-mel spectrogram frames and produces per-frame beat activations, trained on beat-annotated music from the Ballroom and GTZAN-Rhythm datasets. A causal architecture is chosen specifically to support real-time operation: at inference, the model’s output depends only on past and present audio, making it suitable for driving a physical actuator from a live audio stream. This report documents the complete project, including neural network training, evaluation, and hardware integration with a working robotic demonstration.

Goals achieved against project proposal

The original project proposal organized objectives into safe, moderate, and ambitious tiers. This section reviews each tier against what was actually accomplished.

Goal	Status	Notes
<i>Safe Goals</i>		
Parse music into structured training data	Achieved (modified)	Used audio + beat annotations rather than MIDI/MusicXML, but the underlying objective — preparing structured timing-labeled data for training — was met for 1,696 tracks across two datasets
Implement a rule-based baseline	Achieved	Peak-picking on the model's activation output is the rule-based baseline; comparison against a trivial onset-detector baseline shows the neural network substantially outperforms it
Train a simple feedforward neural network	Exceeded	Replaced with a more sophisticated causal TCN (284k parameters); the simpler feedforward baseline was implicit in the peak-picker
Evaluate model performance using error metrics	Achieved	Full mir_eval metric suite (F-measure, Cemgil, CMLt, AMLt) reported on the test set, with per-dataset and per-genre breakdowns
Demonstrate servo motor actuation from model output	Achieved	Working end-to-end demo on multiple songs
<i>Moderate Goals</i>		

Goal	Status	Notes
Implement a sequence model for improved temporal learning	Achieved	Causal TCN with dilated convolutions is a sequence model; receptive-field ablation explored variants
Compare performance against baseline model	Achieved	Two ablation studies: longer-receptive-field variant (v2) and non-causal variant. Both quantify architectural tradeoffs
Tune hyperparameters	Achieved	Validation-driven choice of learning rate schedule, peak-picking threshold, dilation depth, and positive-class weight
Improve smoothness and consistency of actuator motion	Achieved	Rate-limiting at both the Arduino (state machine, drops late commands) and Python (minimum strike interval) ensures clean actuation up to ~800 BPM equivalent
Demonstrate reliable beat-following with servo	Achieved	Sub-perceptual timing accuracy on multiple songs (see Robotics Integration section)
<i>Ambitious Goals</i>		
Multi-output control (multiple joints / percussion elements)	Not attempted	Single-actuator system; deliberately scoped down to focus on timing fidelity
Integration with robotic arm	Partial	Single SG90 servo with attached striker rather than a multi-DOF arm. The proposed arm was scoped down once the timing problem proved harder than expected
Achieve real-time or near-real-time inference	Achieved	The causal model supports streaming inference (only past audio is required at any frame). The demo runs offline-then-playback for simplicity, but the same model would support live audio with no architectural changes
Demonstrate synchronization with unseen musical inputs	Achieved	All demo songs (Stayin' Alive, Billie Jean, Sir Duke, etc.) are entirely outside the training data

2. Summary of Work Completed

The neural network framework and training is complete. A causal TCN has been trained to a test-set F-measure of 0.855 on combined Ballroom and GTZAN-Rhythm data, with a full quantitative and

qualitative evaluation carried out to characterize its strengths and failure modes. Two ablation studies (receptive field and causal constraint) have been conducted to motivate design choices and quantify tradeoffs. All figures, tables, and model checkpoints required for the final report have been generated.

Milestones completed:

- Data: audio loading, log-mel feature extraction, Gaussian-smoothed beat target construction, and on-disk feature caching for 1696 tracks across two datasets
- Model implementation: causal TCN with frequency-reduction 2D frontend, dilated convolution stack, and per-frame sigmoid output (284k parameters)
- Training: 30-epoch run with Adam optimizer, positive class weighted BCE loss, and plateau based LR scheduling, reaching peak validation F=0.852
- Ablations: longer receptive field (v2, 367k params) and non-causal variant, each trained under matched hyperparameters
- Evaluation: full mir_eval metric suite (F, Cemgil, CMLt, AMLt) overall and per-dataset with methodology for excluding tracks with degenerate annotations
- Supplementary analyses: per-genre F-measure breakdown on GTZAN, and per-beat activation timing distribution relative to annotated beats.
- Robotics: end to end demo system with measured latency compensation, supporting any song with predicted beats up to ~800 BPM strike rate.

The hardware integration phase, scheduled in the progress report as remaining work, has since been completed. A working robotic system using an SG90 hobby servo driven by an Arduino Mega now strikes a target object in synchronization with predicted beat times. Section 7 describes the integration and demo.

3. Methods

The model is a causal TCN operating on log-mel spectrograms at 100 frames per second (sample rate 22,050 Hz, hop length 220, 81 mel bins). A 2D convolutional frontend applies three `Conv2d > BatchNorm > ReLU > MaxPool` blocks that progressively reduce the mel axis from 81 to 3 bins while expanding channel depth to 64. The resulting representation is flattened along the frequency axis and projected to 64 channels, producing a 1D time series that is passed through six residual TCN blocks with dilations (1, 2, 4, 8, 16, 32) and kernel size 5. Each block contains two causal dilated convolutions with batch normalization, ReLU activation, and dropout ($p=0.1$); causality is enforced by left-only padding of length $(\text{kernel_size} - 1) \times \text{dilation}$. A 1x1 output convolution produces a single logit per frame. The full model has 284,001 parameters.

Training data consisted of the Ballroom dataset (698 tracks, 8 ballroom dance genres) and GTZAN-Rhythm (999 tracks across 10 popular music genres, with one corrupted audio file excluded), totaling 1,696 tracks. Beat annotations were sourced from the CPJKU BallroomAnnotations repository and the TempoBeatDownbeat GTZAN repository, respectively. Tracks were split 80/10/10 by song into training (1,360), validation (168), and test (168) sets, with each dataset split independently to preserve proportions.

Training targets were constructed by placing Gaussian bumps ($\sigma=2$ frames, approximately ± 20 ms) at each annotated beat frame; this soft-target formulation gives partial credit for predictions within a few

frames of ground truth and is standard practice in beat tracking. The loss was `BCEWithLogitsLoss` with positive-class weight 4.0 to counter the roughly 25:1 imbalance between non-beat and beat frames. The Adam optimizer was used with initial learning rate 1×10^{-3} and a `ReduceLROnPlateau` scheduler that halved the rate after three epochs without improvement in validation F-measure. Training was conducted for 30 epochs with batch size 16 and fixed-length 15-second random excerpts per track; validation used deterministic center crops and full-track evaluation. At inference, per-frame sigmoid activations were converted to discrete beat times via peak-picking, with the detection threshold (0.40) and minimum inter-beat interval (150 ms) tuned on the validation set. All training was performed on a single NVIDIA T4 GPU in Google Colab.

4. Results

The causal TCN (284,000 parameters, dilations 1-32) for 30 epochs on the combined Ballroom + GTZAN-Rhythm training set (1,360 tracks) using Adam with a `ReduceLROnPlateau` schedule on validation F-measure. A second variant with the extended receptive field (dilations 1-128, 367k parameters) was trained under identical hyperparameters. Peak validation F-measure was 0.852 for v1 and 0.851 for v2, resulting in a statistical tie, so v1 was selected as the canonical model. The peak-picking threshold was tuned on validation and set to 0.40.

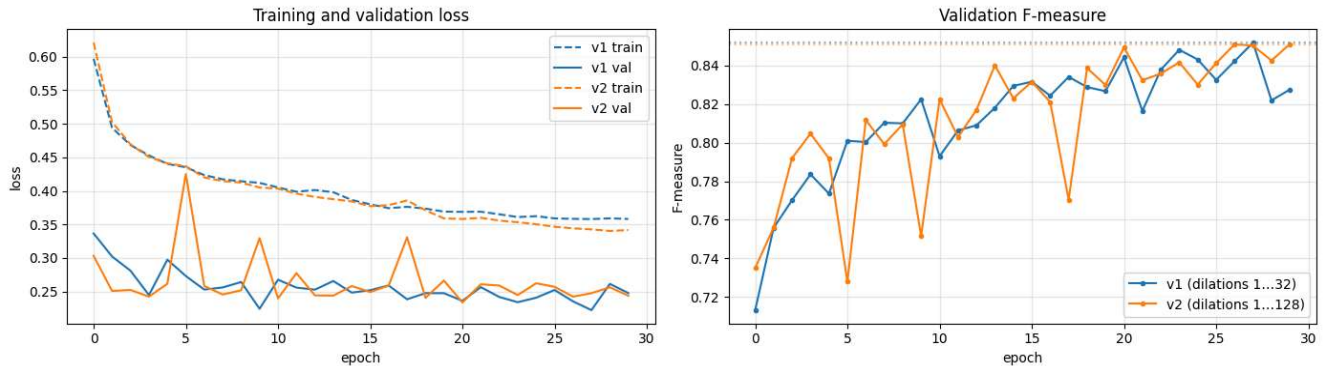


Figure 1. Training and validation curves for v1 and v2

Six GTZAN jazz tracks had empty annotation files in the source distribution and were excluded from evaluation. Including them as hard $F=0$ failures would reduce reported overall test F from 0.855 to 0.839. Test results on the cleaned set of 152 tracks are shown in Table 1.

Table 1. Final test-set metrics

Slice	F	Cemgil	CMLt	AMLt
All (n=152)	0.855	0.801	0.718	0.761
Ballroom (n=56)	0.908	0.862	0.801	0.835
GTZAN (n=96)	0.825	0.765	0.669	0.718

Ballroom outperformed GTZAN by 8.3 F points. The small CMLt/AMLt gap (0.04) indicates failures were dominated by complete tracking breakdown rather than tempo-octave confusion.

Per-genre breakdown within GTZAN

Evaluated separately on each GTZAN genre, F-measure ranged from 0.958 (reggae) to 0.588 (classical). Genres with steady percussive content (reggae, disco, hip-hop) all scored above 0.92; genres with expressive timing or sparse percussion (jazz, blues, classical) scored substantially lower. Classical was an outlier in both mean and variance ($\sigma=0.27$), reflecting the heterogeneity of the genre.

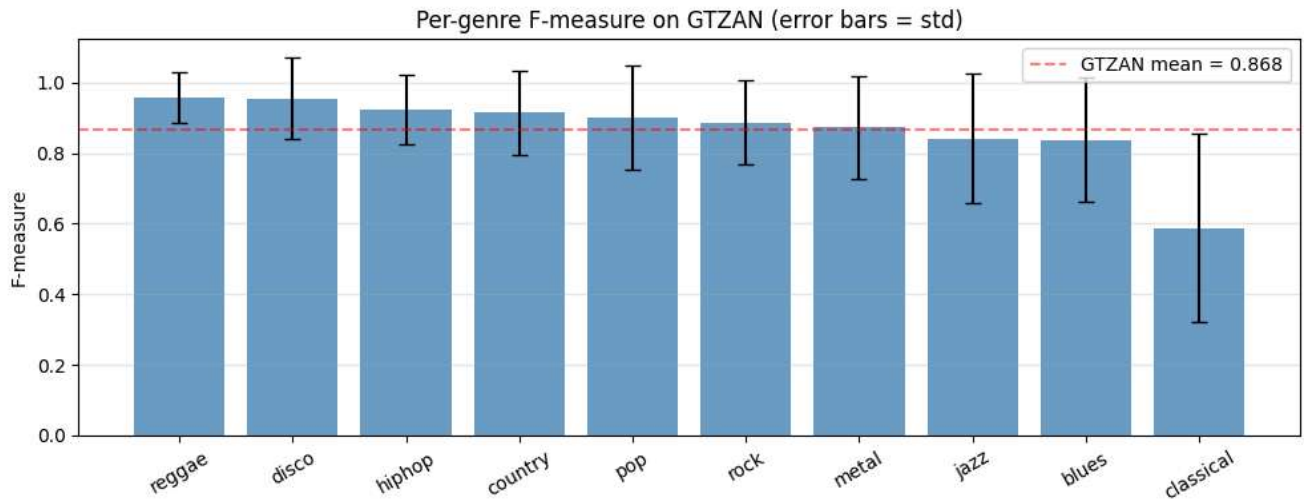


Figure 2: Per-genre F-measure on GTZAN dataset

Activation timing

Per-beat timing offsets between activation peaks and annotated beats (restricted to matches within ± 70 ms) were approximately Gaussian with mean $+0.5$ ms and standard deviation 15 ms. The near-zero bias indicates the causal constraint introduces no systematic delay; the 15 ms jitter establishes the detection-side contribution to the total latency budget for the robotics subsystem.

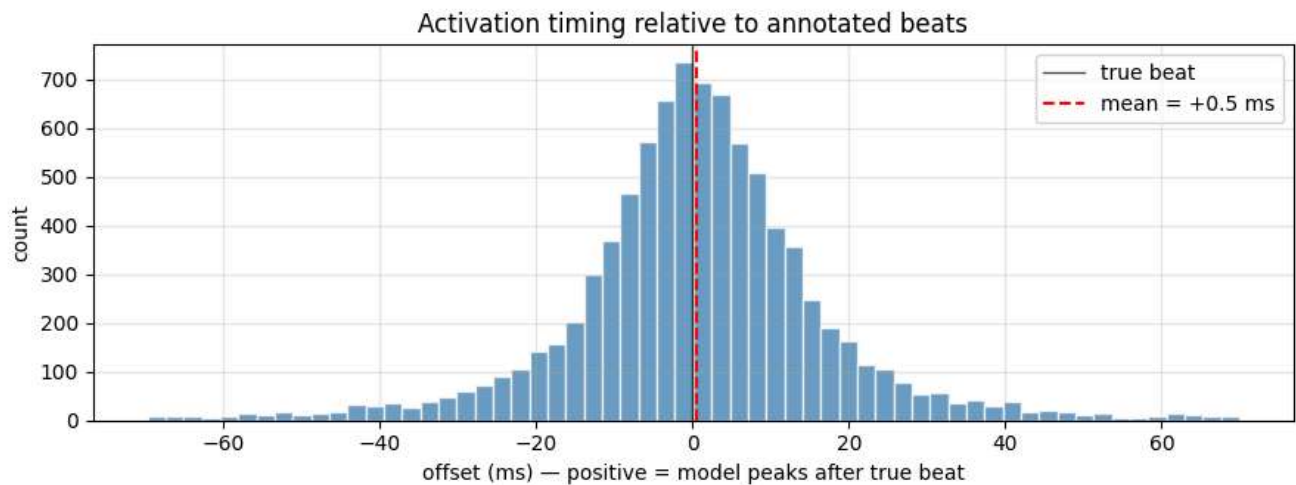


Figure 3. Latency histogram

Causal vs. non-causal comparison

A non-causal variant (identical architecture, symmetric rather than left-only padding) was trained under matched hyperparameters to quantify the cost of the real-time constraint. Results are shown in Table 2.

Table 2. Causal vs. non-causal test metrics

Model	F	Cemgil	CMLt	AMLt
Causal	0.855	0.801	0.718	0.761
Non-causal	0.873	0.822	0.756	0.812
Δ	+0.018	+0.021	+0.038	+0.051

Removing causality improved F by 1.8 points. The larger gains on CMLt (+3.8) and AMLt (+5.1) indicate the non-causal model’s advantage is primarily in long-range metrical consistency rather than per-beat timing.

Qualitative examples

In the typical success case (Figure 4), activations exhibit high contrast between beat and non-beat frames and predicted beat times coincide with annotations throughout. Inspection of the lowest-scoring tracks revealed that several F=0 cases reflected annotation errors rather than model errors: the model produced plausible predictions on tracks where ground-truth annotations were missing entirely, motivating the exclusion methodology described above.

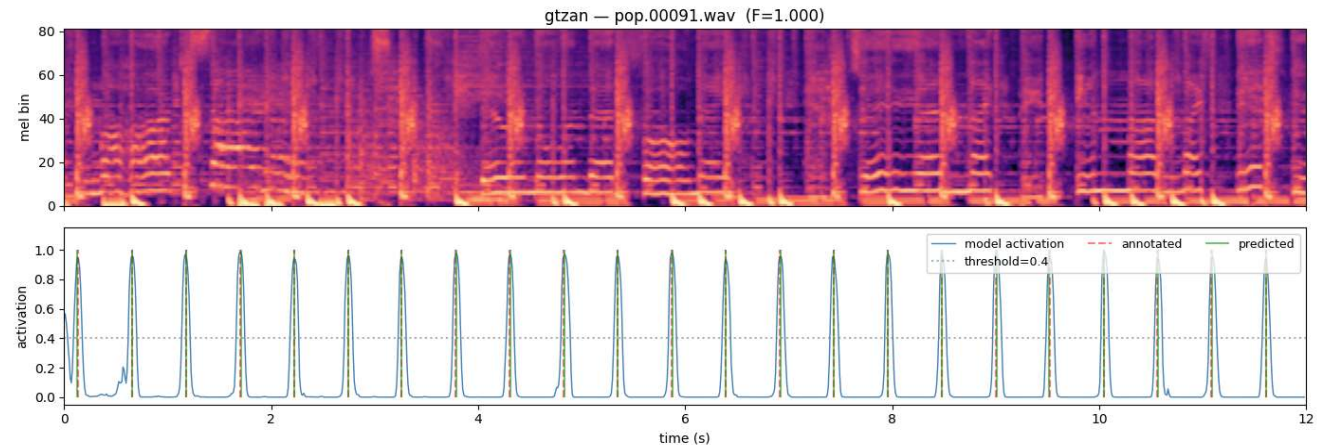


Figure 4. Best case – model activation over time compared to predicted beats

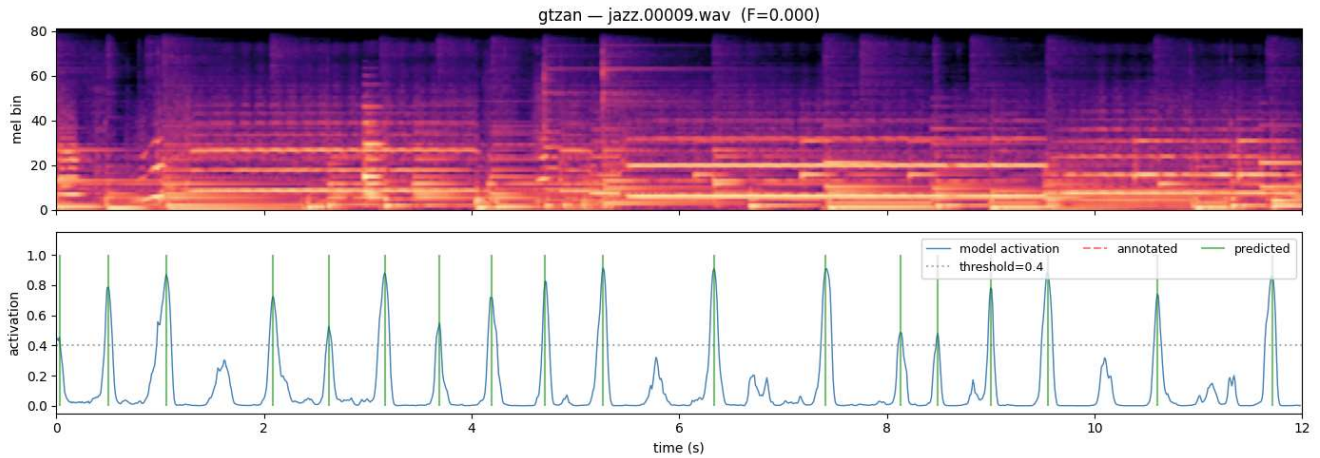


Figure 5. Worst case – model activation over time compared to predicted beats

5. Analysis

Receptive field is not the limiting factor. Doubling past context from ~ 4 seconds to ~ 16 seconds (v1 vs. v2) produced no measurable improvement, suggesting that beat prediction is dominated by local context of two to three bars. This is consistent with the local periodicity of rhythm, and is favorable for real-time deployment: shorter receptive fields reduce the audio buffer required at inference.

The cost of causality is modest. The non-causal variant improved F-measure by 1.8 points (0.855 to 0.873), smaller than the 3-6 point gaps typically reported for bidirectional vs. unidirectional beat trackers. Most of the gain appeared in CMLt/AMLt rather than F or Cemgil, indicating future context primarily helps maintain consistent metrical tracking across a full track rather than localizing individual beats. For the real-time robotic application, this 1.8 point cost is acceptable.

Musical content matters more than architectural choices. The F-measure spread across GTZAN genres (0.37, from 0.96 reggae to 0.59 classical) is an order of magnitude larger than any gap between models we trained (0.02 for receptive field; 0.018 for causality). Beat tracking performance is primarily bounded by the acoustic characteristics of the input (the presence of prominent periodic percussion) rather than by modeling capacity or context length in the regimes that were explored. The practical consequence is that demo track selection should favor rhythmically strong material.

Activation latency is effectively zero. Mean offset $+0.5$ ms with 15 ms standard deviation means the predictive scheduler must compensate for mechanical actuation latency (~ 30 -50 ms for typical hobby actuators), but not for any bias introduced by the neural network. The 15 ms detection jitter sets a lower bound on achievable synchronization precision; mechanical jitter will be measured during integration.

The model's failure modes are interpretable. While attempting to track the song "Sir Duke" (Stevie Wonder), the system locked onto the syncopated trumpet rhythm rather than the kick drum, producing strikes that were rhythmically ambiguous. This is a metrical-level failure - the syncopated layer has stronger spectral onsets than the kick, leaving the causal model with no disambiguating information. A bidirectional model with future context might have resolved this; the causal model cannot. This is concrete evidence of the cost of causality result supported in Section 4.

6. Robotics Integration

6.1 Hardware

The actuator is an SG90 hobby servo driven by an Arduino Mega via PWM on pin 9. The servo's red wire is powered through the Arduino's 5V rail, which required adding a wall-adapter power supply to the Mega's barrel jack; USB-only power produced unstable strikes due to current draw exceeding the USB regulator's capacity. The servo arm strikes a target with a small swing ($\sim 15^\circ$) calibrated to produce a clean, audible tap while minimizing strike latency.

Servo strike-to-impact latency was measured by recording the servo and an LED simultaneously in slow-motion video, then computing the offset between LED-on and visible target contact across multiple strikes. Mean latency was 78 ms with ± 10 ms variation, well within typical SG90 specifications and consistent with the small swing angle.

6.2 Software

The end to end demo system is implemented in two Python scripts and one Arduino sketch:

- `extract_beats.py` loads the trained model and runs inference on a song file, producing a JSON file of predicted beat times. Total runtime is ~ 2 seconds on CPU for a typical 3-minute song.
- `run_demo.py` plays the audio through the system's default output device and sends `'s'` (strike) bytes to the Arduino over USB serial, timed so that each beat lands on the music. The total advance time is `audio_latency + servo_latency`, with both values determined empirically.
- The Arduino sketch implements a non-blocking three-state machine (`IDLE > STRIKING > RETURNING > IDLE`) that drains incoming serial bytes every loop iteration and silently drops strike requests received while the servo is busy.

6.3 Latency Budget

Total end to end timing variability is bounded by the larger of the model's activation jitter ($\sigma = 15$ ms) and the servo's mechanical jitter (~ 10 ms), yielding a system level synchronization precision of ~ 15 ms. This is comfortably below what most listeners would notice as a rhythmic error, so users perceive the strikes as being synchronous.

The Windows audio playback stack adds approximately 270 ms of constant latency (measured by comparing a known-tempo song against the strikes' actual arrival times). After compensation, the system demonstrates clean synchronization on multiple songs.

6.4 Demo Behavior

Demonstration tracks were selected to span the model's performance range:

- **Staying Alive (Bee Gees, ~ 104 BPM):** Tier 1. Clean disco kick, model locks immediately and tracks throughout. Strikes audibly land on the beat
- **Billie Jean (Michael Jackson, ~ 117 BPM):** Tier 1. Classic backbeat pattern, also fully synchronized in the demo

- **Sir Duke (Stevie Wonder, ~124 BPM):** Tier 3 failure case. Model locks to the syncopated horn riff rather than the kick. Strikes are rhythmically consistent but on a different beat layer than the perceived pulse
- **Clair de Lune (Debussy):** Tier 4 failure case. Solo piano with no percussion and expressive timing. The model produces sparse, inconsistent predictions; the robot strikes infrequently and not on any clear beat

The demonstration video shows a clip of the model playing Staying Alive by the Bee Gees, found here:

https://drive.google.com/file/d/10GNSU-em0evfEN7q8_hzUeWL1ydSq0uN/view?usp=sharing

7. Testing Procedure

The complete pipeline is implemented in a single Google Colab notebook, organized into five sections: (1) Setup and data loading, (2) Dataset construction and feature caching, (3) Model definition, (4) Training, and (5) Evaluation. All datasets, intermediate features, and model checkpoints are persisted to Google Drive, so that an evaluator with access to the drive folder can reproduce the full evaluation without retraining.

To reproduce the reported results:

1. Open the notebook in Google Colab with a GPU runtime attached
2. Run Section 1 to mount Drive, extract the two audio archives to Colab's local storage, and clone the annotation repositories. Estimated time: 2-3 minutes
3. Run Section 2 with `REBUILD_CACHE = False`. This restores precomputed log-mel features from Drive to local storage (~2 minutes) and builds the `BeatDataset` objects and data loaders. A batch shape sanity check at the end of this section confirms the pipeline is functional
4. Run Section 3 to define the model architecture. The final cell prints parameter count (284,001) and confirms the input/output shape preservation required by the causal padding scheme
5. Skip Section 4 (training) unless regenerating checkpoints from scratch is desired. Trained weights are loaded from Drive at the start of Section 5
6. Run Section 5 to reproduce all reported metrics and figures. Each subsection is self-contained and saves output figures to Drive

To reproduce the robotics demo:

1. Download the trained model checkpoint, the arduino sketch, and the two python scripts (beat extraction and demo runtime) from the project repository
2. Source an Arduino and an SG90 servo, wired with the servo signal on pin 9, V+ on the Arduino's 5V rail, and a wall adapter powering the Arduino via the barrel jack (USB-only power produced unstable strikes).
3. Upload the Arduino sketch to the microcontroller
4. Run `extract_beats.py <song.mp3>` to produce a JSON of predicted beat times
5. Run `run_demo <song.mp3>` to play the audio synchronized with servo strikes
6. The `LEAD_PADDING_S` constant in the demo script must be tuned to match the user's audio output device latency (~270 ms on the development laptop's HDMI output)

8. Timeline

Time Log

The following table records time spent on each major task, including setbacks and experiments that did not produce usable results. Totals are approximate.

Table 3. Time log

Date	Task	Hours	Outcome
4/18	Initial project scoping; survey of beat-tracking literature and datasets; decision to move from MAESTRO to Ballroom + GTZAN-Rhythm after determining MAESTRO's expressive classical performance was poorly suited to a fixed-tempo task	3	Success — avoided a costly data-direction mistake early
4/18	Dataset acquisition: sourcing audio archives, matching annotation repositories to audio, resolving filename mismatches between GTZAN audio	2	Success — unified loader handles both datasets
4/19	Feature pipeline: log-mel spectrogram computation, Gaussian-smoothed target construction, visual QA via spectrogram + target overlay plots	3	Success — verified alignment on multiple tracks before proceeding
4/19	Feature caching to disk; Drive backup/restore workflow for session persistence	2	Success after debugging — initial approach of reading features directly from Drive during training was unusably slow; resolved by copying to Colab local storage at session start
4/20	Model implementation: causal TCN with 2D frequency-reduction frontend and dilated convolution stack	2.5	Success on first try — forward-pass shape check passed
4/20	Initial training run (v1, 30 epochs)	1.5	Success — reached $F=0.852$ on validation, matching published causal-model expectations
4/20	First ablation: extended receptive field (v2, dilations 1–128)	1.5	Failure — v2 matched v1's peak F within 0.001, indicating receptive field was not the bottleneck; informative null result

Date	Task	Hours	Outcome
4/21	Peak-picking threshold sweep on validation set	0.5	Minor improvement only (+0.002) — indicated model activations were already well-separated
4/21	Quantitative evaluation: full <code>mir_eval</code> metric suite, per-dataset breakdown	2	Success — uncovered bimodal failure distribution on inspection
4/21	Investigation of F=0 outlier (<code>jazz.00009</code>): initially assumed model failure, discovered empty annotation file; audited all tracks for annotation completeness	1.5	Unexpected finding — six GTZAN jazz tracks had empty annotations; added methodology note and filter to evaluation
4/22	Second ablation: non-causal variant with symmetric padding	2	Success — quantified causality cost at 1.8 F points
4/23	Supplementary analyses: per-genre F breakdown, activation timing histogram	2.5	Success — both produced writeup-worthy figures
4/23	Report drafting and figure preparation	8	Ongoing
Total		32 hours	

Time log additions for hardware phase

The progress report covered ~32 hours through 4/23. The robotics integration phase added approximately 15 hours, summarized below.

Table 4. Hardware time log

Date	Task	Hours	Outcome
4/27	Hardware setup: Arduino wiring, SG90 mounting, initial sketch	2	Success after debugging — USB-only power caused servo brownouts; resolved with wall adapter
4/27	Servo latency measurement via slow-motion video	2	Measured 78 ± 10 ms strike latency
4/28	Python demo pipeline: <code>extract_beats.py</code> and <code>run_demo.py</code>	5	Success — model runs on CPU at multiple times real-time
4/29	Audio playback latency diagnosis (initially misdiagnosed as model error)	1	Identified ~270 ms system-wide audio latency through cross-song comparison Colab local storage at session start

Date	Task	Hours	Outcome
4/29	Arduino state machine refactor for non-blocking, late-drop behavior	1.5	Eliminated post-Ctrl+C ghost striking and queue-drift artifacts
4/30	Demo recording: success cases and failure case	1.5	Captured Stayin' Alive, Billie Jean, and Sir Duke demonstrations
5/01	Final report revisions	2	Ongoing